COMP 597 – Energy Profiling of Whisper Fine-Tuning: Instrumentation Overhead and Resource Utilization

Bo Lau
(261026663)
McGill University

1 Introduction

Training workloads are increasingly evaluated not only by accuracy and throughput but by the energy and carbon cost of running them at scale. This project adopts that perspective and studies a fine-tuning scenario derived from the Milabench benchmarking stack [1], using reported CO_2 equivalents from CodeCarbon [2] as a primary lens on "where the carbon goes" alongside detailed timing and utilization.

Milabench-style setups typically rely on synthetic, in-memory data paths that keep the accelerator busy and abstract away storage. Here we pair that view with a complementary configuration in which fixed-shape Mel spectrograms reside on local SSD and are fed through a conventional PyTorch `Dataset` and `DataLoader`. The motivating question was whether moving work off a purely RAM-backed pipeline and onto explicit disk reads would reveal input bottlenecks or qualitatively different energy profiles—for example, if sustained I/O or CPU-side preparation became a first-order concern.

In practice, the behavior we observe is closer to a classical compute-bound regime. Once tensors are resident in the Linux page cache, repeated access from disk is difficult to distinguish from memory-backed serving for this workload, and GPU utilization remains high during steady-state training. The sections that follow detail the Whisper fine-tuning setup, the instrumentation tiers we used to separate measurement overhead from real compute time, and how energy concentrates in the backward pass and GPU power draw despite the two data backends.

2 Workload Description

Model and Task. We fine-tune OpenAI's `whisper-tiny` model [3] for audio classification using the Hugging Face `WhisperForAudioClassification` architecture. The model is a Transformer-based encoder–decoder with ∼39M parameters. We attach a linear classification head and train end-to-end with the AdamW optimizer at a learning rate of 10^{-6} on synthetic audio data. No learning rate schedule or weight decay is applied.

Data. We evaluate two data-loading backends that differ fundamentally in where data resides and how it reaches the GPU:

- **Disk Cache**: The full synthetic dataset (5,500 Mel spectrograms) is generated once and saved to a `.pt` file on local SSD. Every subsequent training run loads this file through a standard PyTorch `Dataset` and `DataLoader`, avoiding regeneration. This path exercises the OS storage stack and is subject to Linux page-cache behavior: after the first epoch the file typically resides in RAM-backed page cache, making later reads indistinguishable from memory access.
- **In-Memory (Milabench)**: A small batch of samples is created in RAM and repeated with a configurable repeat factor to reach 16,000 logical samples—long enough for a meaningful training run without ever writing or reading a file. Because the data lives entirely in memory, there is no disk I/O and therefore no storage bottleneck to optimize.

Both backends produce fixed-shape Mel spectrograms matching Whisper's expected input format (80 frequency bins × 3000 time steps).

Hardware. All experiments run on a single SLURM-allocated node (`gpu-grad-01`) with:

- **GPU**: NVIDIA RTX A5000 (24 GB GDDR6, 234W TDP)
- **CPU**: AMD EPYC 9554 64-Core (4 cores allocated via SLURM)
- **RAM**: 4 GB (SLURM job allocation)
- **OS**: Linux 5.15, Python 3.14, PyTorch with CUDA

Terminology. A *step* consists of one forward pass, one backward pass, and one optimizer update on a single batch. Each *phase* (forward, backward, optimizer) is timed individually. Training proceeds for one epoch or until a 5-minute wall-clock limit, whichever comes first.

3 Methodology

3.1 Experiment Design

Each experiment is a 5-minute training run repeated 3 times to account for system variability. We sweep three batch sizes: 32, 64, and 128—the largest power-of-2 that fits in GPU memory and two smaller values. For both data backends, we test 0 and 4 `DataLoader` workers.

With 0 workers, data loading occurs on the main training thread; with 4, background processes prepare batches in parallel.

The number of training steps per run depends on both the dataset size and the time limit:

- **Disk**: one epoch = $\lceil 5500/\text{batch_size} \rceil$ steps. At batch 32, this yields 172 steps (∼72 s with noop), well within 5 minutes.
- **Milabench**: one epoch = 16000/batch_size steps. At batch 32, this yields 500 steps (∼211 s with noop), also within 5 minutes.

3.2 Measurement Setup

We employ six *trainer stats* modules that instrument the training loop with increasing levels of detail:

- **noop**: All hooks are empty (`pass`). Baseline for end-to-end wall-clock time with zero instrumentation overhead.
- **simple**: CUDA-synchronized timers before and after each phase. Records forward, backward, and optimizer durations. No resource monitoring or file I/O during training.
- **phase_times**: Inherits `simple`'s timing hooks; writes per-step phase durations to a CSV file at the end of training.
- **resource_util**: Records GPU utilization, GPU power draw, GPU memory (allocated and reserved), CPU utilization, RAM usage, and disk I/O per step via NVML and `psutil`. Also includes phase timing via CUDA sync.
- **codecarbon**: CodeCarbon energy tracking with a 500 ms power sampling interval, combined with phase timing.
- **codecarbon_e2e**: CodeCarbon with 86,400 s sampling interval (effectively one sample for the entire run), designed to minimize polling overhead while still capturing total energy.

GPU Timing Synchronization. All timing-capable modules (every module except `noop`) call `torch.cuda.synchronize(device)` before and after each phase boundary—8 calls per step total. This ensures that asynchronously dispatched CUDA kernels have completed before timestamps are recorded. Without synchronization, the CPU-side timer would only measure kernel *launch* time, not actual GPU execution time. However, this serializes the GPU pipeline: the CPU cannot enqueue the next operation until the previous one completes, which prevents overlapping data loading with GPU compute.

Energy Measurement. CodeCarbon samples GPU power via NVML at the configured interval. CPU power is estimated from the processor's TDP rather than measured directly, as RAPL counters were not accessible on the cluster (permission denied). Total energy is reported as the sum of GPU, CPU, and RAM energy. CO_2 emissions are computed using CodeCarbon's grid carbon intensity factor for Quebec, Canada.

4 Results

We organize results into five categories: end-to-end timing, energy consumption, per-phase analysis, system utilization, and instrumentation overhead. All figures referenced in this section are collected in Figure 1.

4.1 End-to-End Execution Time

Table 1 shows the mean training duration for each trainer stats module. With disk loading and `workers=0`, all instrumented trainers are 30–36% slower than noop. Crucially, this overhead is nearly identical across `simple`, `phase_times`, and `resource_util`—modules that differ substantially in what they measure but all share the same `torch.cuda.synchronize()` calls. This uniformity demonstrates that the overhead originates from CUDA synchronization, not from the monitoring logic itself.

Table 1: Mean training duration (seconds), averaged over 3 runs.

Trainer	B=32	B=64	B=128
Disk, workers=0			
noop	72.0	71.3	70.3
simple	95.7	94.3	92.3
phase_times	96.0	93.7	94.3
resource_util	98.0	95.3	94.3
codecarbon	98.0	93.7	91.7
Disk, workers=4			
noop	114.5	112.7	116.3
simple	115.7	115.7	117.7
resource_util	119.3	119.0	124.3
codecarbon	118.7	117.0	121.0
Milabench, workers=0			
noop	211.0	208.7	205.0
simple	213.3	214.0	210.7
resource_util	214.7	214.3	211.0
codecarbon	217.0	215.7	211.7
Milabench, workers=4			
noop	224.7	213.0	210.0
simple	231.7	217.3	215.0
resource_util	231.3	217.7	215.0
codecarbon	238.7	219.0	215.7

When comparing against `simple` as a baseline (isolating monitoring overhead from sync overhead), resource monitoring adds <3% and CodeCarbon adds 1–6% on top of the sync cost.

The configuration dependence of overhead is explained by the degree of CPU/GPU overlap lost to synchronization. With `workers=0` on disk, the main thread can load the next batch *while* the GPU processes the current one—sync destroys this overlap. With `workers=4`, background

processes keep batches ready regardless of main-thread blocking. With Milabench, data generation is inherently slower (∼1.2 s/step vs ∼0.4 s/step for disk), so the GPU is already waiting for data and sync adds little.

4.2 End-to-End Energy Consumption

Table 2 summarizes CodeCarbon energy measurements. Across all configurations, energy breaks down as approximately 66% CPU, 32% GPU, and 2% RAM. The CPU-dominated breakdown is an artifact of TDP-based estimation; the actual CPU power draw is likely much lower than the reported ∼360 W estimate across all 64 cores, given that only 4 cores are allocated.

Table 2: Mean energy consumption (kWh) from CodeCarbon, averaged over 3 runs.

Source	W	B=32	B=64	B=128
Total energy (kWh)				
Disk	0	0.0142	0.0138	0.0136
Disk	4	0.0161	0.0160	0.0165
Mila.	0	0.0328	0.0330	0.0327
Mila.	4	0.0354	0.0334	0.0331
GPU energy only (kWh)				
Disk	0	0.0042	0.0042	0.0042
Disk	4	0.0039	0.0040	0.0041
Mila.	0	0.0106	0.0110	0.0110
Mila.	4	0.0109	0.0109	0.0110

Disk experiments consume roughly half the energy of Milabench experiments, proportional to their shorter duration. Energy is relatively stable across batch sizes since runs are time-limited: larger batches complete fewer steps, but each step processes more data. CO_2 emissions are very low (<0.15 g per run) due to Quebec's hydroelectric grid.

4.3 Per-Phase Timing Analysis

Figure 1a shows per-phase mean durations for batch 32. The backward pass dominates at ∼290 ms (∼68% of step time), followed by forward at ∼125 ms (∼30%), with the optimizer step negligible at ∼2 ms (∼0.5%). The backward pass is ∼2.3× slower than the forward pass because it computes gradients for all parameters in addition to traversing the computation graph. The optimizer step is fast because AdamW performs only element-wise updates.

As batch size increases from 32 to 128, both forward and backward durations increase (more data per step), but the optimizer step remains constant since it depends only on the number of parameters, not the batch size. The standard deviation across steps is low (<5 ms for steady-state steps), indicating stable per-step performance after the first warm-up step.

4.4 System Utilization

Figures 1d and 1g show GPU utilization and full resource timelines. GPU utilization is consistently above 95% during steady-state training across all batch sizes, confirming that the workload is compute-bound on the RTX A5000. Step 1 exhibits lower utilization (∼1–6%) due to CUDA JIT compilation and memory allocation, but this warm-up cost is amortized over the full training run.

CPU utilization averages ∼120% across the 4 allocated cores, reflecting the main PyTorch training thread plus data loading. GPU memory usage scales modestly with batch size: ∼6.8 GB for batch 32, ∼6.9 GB for batch 64, and ∼7.2 GB for batch 128—all well within the 24 GB A5000 capacity.

Disk I/O is concentrated at the beginning of training (initial data loading) and is negligible during steady-state steps, confirming that the data fits in the OS page cache after the first pass.

4.5 Batch Size Effects

The number of steps completed per run varies inversely with batch size: 172 / 86 / 43 steps for batch 32 / 64 / 128 on disk data, and 500 / 250 / 125 on Milabench. Despite fewer steps, larger batches achieve higher throughput (samples/s) because GPU utilization per step improves with larger workloads.

Total energy is approximately constant across batch sizes (Table 2) because all runs are bounded by the same 5-minute time limit. The energy-per-sample is thus lower for larger batches. However, in practice, the choice of batch size also affects convergence and final model quality—considerations outside the scope of this profiling study.

4.6 Instrumentation Overhead

Figures 1h and 1i compare the overhead of each trainer stats module relative to noop for `workers=0` and `workers=4`, respectively. The overhead hierarchy is:

1. **CUDA synchronization** (shared by all non-noop trainers): 30–36% with disk/`workers=0`; 1–5% with `workers=4`; 1–5% with Milabench.
2. **Resource monitoring** (NVML, psutil, disk I/O): <3% additional overhead beyond simple timer-based sync.
3. **CodeCarbon energy tracking**: 1–6% additional overhead, with `codecarbon_e2e` (sparse sampling) slightly lower than `codecarbon` (500 ms polling).

The stark difference between disk/`workers=0` (30–36% overhead) and all other configurations (1–7%) arises from the loss of CPU/GPU pipeline overlap. With `workers=0`, the main thread loads data asynchronously with GPU execution; CUDA synchronization blocks this overlap, adding ∼20 s to a 72 s run. With background workers or slower data pipelines, there is less overlap to lose.

5 Discussion

Where is energy spent? The backward pass dominates compute time at ∼68% of each step, making gradient computation the primary driver of GPU energy consumption. In terms of CodeCarbon's reported total, CPU energy appears to dominate (∼66%), but this is an artifact of TDP-based estimation: CodeCarbon attributes the full 360 W TDP of the 64-core EPYC to the process even though only 4 cores are allocated. The actual CPU power draw is likely 20–40 W, which would make GPU energy the true dominant component at >70% of total.

GPU utilization and inefficiency. GPU utilization exceeds 95% during steady-state training for all batch sizes, confirming the workload is compute-bound. The primary source of inefficiency is the first training step, which takes ∼2× longer due to CUDA JIT compilation and initial memory allocation. For short runs (5 minutes), this warm-up represents <2% of total time. No significant underutilization periods were observed during steady-state training, suggesting that Whisper fine-tuning on synthetic data saturates the A5000's compute capacity.

Trade-offs: throughput, batch size, and energy. Larger batch sizes increase per-step compute time but reduce the number of steps needed, yielding higher throughput (samples/s). Since our runs are time-limited, total energy is approximately constant across batch sizes. In fixed-epoch training (rather than fixed-time), larger batches would reduce total energy by completing the epoch in fewer, more GPU-efficient steps. The choice involves a trade-off with convergence behavior that is beyond our profiling scope.

Measurement overhead. The dominant overhead is `torch.cuda.synchronize()`, which is *required* for accurate per-phase GPU timing. This is not a limitation of the monitoring tools but a fundamental property of CUDA's asynchronous execution model. The practical impact depends on pipeline structure: with background DataLoader workers (`workers=4`), overhead is <7% across all configurations—acceptable for profiling runs. For production training where phase-level timing is not needed, the `noop` stats module provides zero overhead.

Data loading: disk cache vs. in-memory. The two backends differ in three respects: *where* data lives, *how* it is served, and what that implies for I/O bottlenecks.

With the disk cache backend, the full dataset is serialized to a `.pt` file on SSD before training begins. Each training run loads this file via a standard PyTorch `DataLoader`. On the first access, data travels from SSD into the Linux page cache; on every subsequent access it is served directly from RAM-backed page cache. Because the dataset (5,500 samples) fits comfortably in the 4 GB SLURM allocation, steady-state disk I/O is effectively zero after warm-up (confirmed in Figure 1g). The disk path with `workers=0` completes one epoch in ∼72 s (172 steps).

The in-memory (Milabench) backend takes a fundamentally different approach: a small batch of synthetic samples is generated once in RAM and repeated to reach the target dataset size (16,000 samples). No file is ever written or read, so the OS storage stack is bypassed entirely. Per-step data delivery is slightly slower (∼1.2 ms/step vs. ∼0.4 ms/step for disk) because samples are regenerated from the in-memory template each iteration, yielding ∼211 s for 500 steps. Because the data never leaves RAM, the Milabench path has no I/O bottleneck by construction—the GPU is fed as fast as memory bandwidth allows, which keeps utilization near 100% throughout the run.

However, this in-memory design is not representative of most production training workloads. Real-world datasets (e.g., hours of raw audio, high-resolution images, or large text corpora) are typically too large to fit in RAM and must be streamed from disk or networked storage. In those settings, I/O can become a first-order bottleneck—especially when storage throughput cannot keep pace with GPU compute. The Milabench approach is therefore useful as an upper bound on GPU utilization, isolating compute behavior from storage effects, but it should not be taken as evidence that data loading is unimportant in practice.

Despite these architectural differences, the two backends produce nearly identical GPU utilization (>95%) and per-step phase-time distributions during steady state in our experiments. This is because our synthetic dataset is small enough to fit in the Linux page cache, so even the disk path behaves like an in-memory path after warm-up. The practical consequence is that for small, fixed-shape datasets on modern hardware, data loading strategy has negligible impact on carbon cost per sample. Workloads with larger or more complex datasets would likely show a different picture.

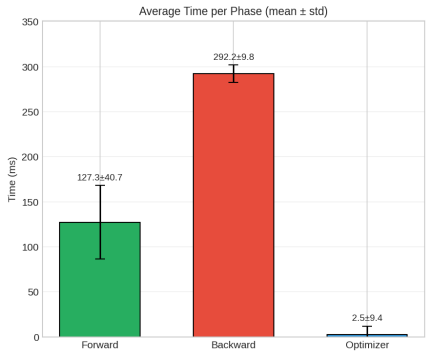
Limitations. (1) CPU energy is TDP-estimated, not measured via RAPL, so the reported CPU/GPU energy split is unreliable. (2) Training duration is parsed from tqdm with integer-second precision; sub-second accuracy would require additional high-resolution timers. (3) All experiments use synthetic data; real audio datasets with variable-length inputs and I/O-bound loading patterns may exhibit different utilization profiles. (4) We profile a single GPU; multi-GPU training would introduce communication overhead not captured here.

References

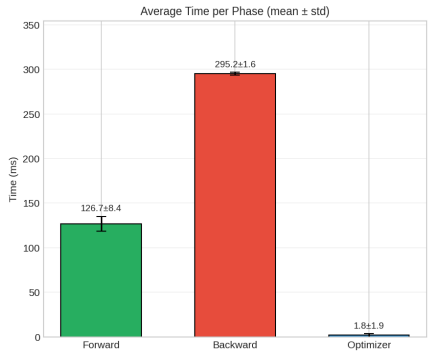
[1] Milabench contributors, "Milabench," GitHub repository, `https://github.com/milabench/milabench`.

[2] CodeCarbon contributors, "CodeCarbon: Track emissions from computing," `https://github.com/mlco2/codecarbon`.

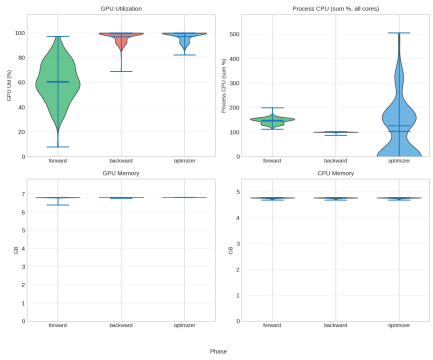
[3] A. Radford et al., "Robust Speech Recognition via Large-Scale Weak Supervision," *Proc. ICML*, 2023.



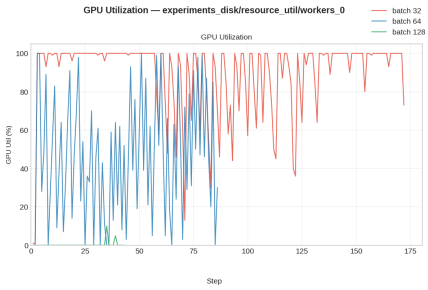
(a) Phase timing (disk, w=0, B=32).



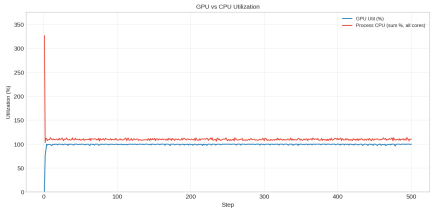
(b) Phase timing (Milabench, w=0, B=32).



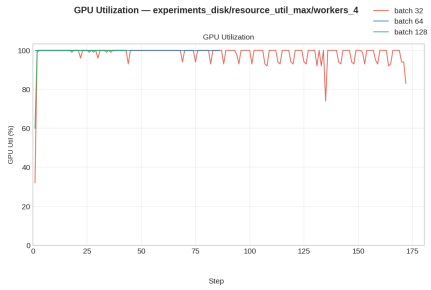
(c) Per-phase resource violins (disk, w=0, B=32).



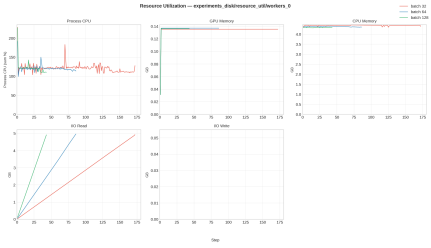
(d) GPU util. across batch sizes (disk, w=0).



(e) GPU and CPU util. (Milabench, w=0, B=32).

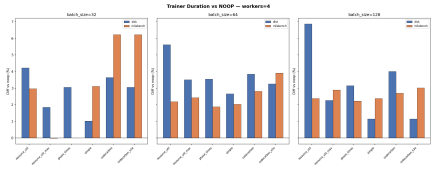


(f) GPU util. across batch sizes (disk, w=4).



(g) Full resource timeline (disk, w=0).

(h) Instrumentation overhead (workers=0).



(i) Instrumentation overhead (workers=4).

Figure 1: Experimental results. **Row 1**: Per-phase timing shows backward pass dominating at ∼68%; violin plots confirm GPU util. drops during forward and spikes during optimizer. **Row 2**: GPU utilization exceeds 95% at steady state for both data backends; workers=4 introduces periodic CPU spikes from background loaders. **Row 3**: Full resource timeline shows stable utilization after warm-up; instrumentation overhead is 30–36% with workers=0 (dominated by CUDA sync) but only 1–7% with workers=4.